```
from google.colab import files
uploaded = files.upload()  # select uber.csv
```

Choose files uber.csv
uber.csv(text/csv) - 23458139 bytes, last modified: 04/02/2026 - 100% done
Saving uber.csv to uber (1).csv

```
import pandas as pd  # for data handling

df = pd.read_csv("uber.csv")  # change name if needed
df.head()
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	4
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	4
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	4

```
# pandas already imported in previous cell, reuse here

print(df.shape)
print(df.info())
print(df.describe(include="all"))
print(df.isna().sum())
```

```
 3   pickup_datetime    200000 non-null  object
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB
None
          Unnamed: 0                        key    fare_amount  \
count   2.000000e+05                     200000  200000.000000
unique           NaN                     200000            NaN
top              NaN  2009-08-29 02:36:56.0000003            NaN
freq             NaN                          1            NaN
mean    2.771250e+07                        NaN      11.359955
std     1.601382e+07                        NaN       9.901776
min     1.000000e+00                        NaN     -52.000000
25%     1.382535e+07                        NaN       6.000000
50%     2.774550e+07                        NaN       8.500000
75%     4.155530e+07                        NaN      12.500000
max     5.542357e+07                        NaN     499.000000

                pickup_datetime  pickup_longitude  pickup_latitude  \
count                    200000     200000.000000    200000.000000
unique                   196629               NaN              NaN
top     2009-02-12 12:46:00 UTC               NaN              NaN
freq                         4               NaN              NaN
mean                       NaN        -72.527638        39.935885
std                        NaN         11.437787         7.720539
min                        NaN      -1340.648410       -74.015515
25%                        NaN        -73.992065        40.734796
50%                        NaN        -73.981823        40.752592
75%                        NaN        -73.967154        40.767158
max                        NaN         57.418457      1644.421482

        dropoff_longitude  dropoff_latitude  passenger_count
count       199999.000000     199999.000000    200000.000000
unique               NaN               NaN              NaN
top                  NaN               NaN              NaN
freq                 NaN               NaN              NaN
mean           -72.525292         39.923890         1.684535
```

```
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    1
dropoff_latitude     1
passenger_count      0
dtype: int64
```

```
# using pandas only

cols_to_drop = [c for c in ["Unnamed: 0", "key"] if c in df.columns]
df = df.drop(columns=cols_to_drop)

df = df.dropna(subset=[
    "fare_amount", "pickup_datetime",
    "pickup_longitude", "pickup_latitude",
    "dropoff_longitude", "dropoff_latitude",
    "passenger_count"
])

print(df.shape)
```

```
(199999, 7)
```

```
# pandas reused

df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"], errors="coerce")
df = df.dropna(subset=["pickup_datetime"])

df["pickup_year"] = df["pickup_datetime"].dt.year
df["pickup_month"] = df["pickup_datetime"].dt.month
df["pickup_day"] = df["pickup_datetime"].dt.day
df["pickup_hour"] = df["pickup_datetime"].dt.hour
df["pickup_weekday"] = df["pickup_datetime"].dt.weekday

df[["pickup_datetime", "pickup_year", "pickup_month",
    "pickup_day", "pickup_hour", "pickup_weekday"]].head()
```

	pickup_datetime	pickup_year	pickup_month	pickup_day	pickup_hour	pickup_weekday
0	2015-05-07 19:52:06+00:00	2015	5	7	19	3
1	2009-07-17 20:04:56+00:00	2009	7	17	20	4
2	2009-08-24 21:45:00+00:00	2009	8	24	21	0
3	2009-06-26 08:22:21+00:00	2009	6	26	8	4
4	2014-08-28 17:47:00+00:00	2014	8	28	17	3

```
import numpy as np  # for numerical operations

# Haversine distance (km)
def haversine_distance(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2.0) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2.0) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    r = 6371
    return c * r

df["distance_km"] = haversine_distance(
    df["pickup_latitude"],
    df["pickup_longitude"],
    df["dropoff_latitude"],
    df["dropoff_longitude"],
)

df[["fare_amount", "distance_km"]].head()
```

	fare_amount	distance_km
0	7.5	1.683323
1	7.7	2.457590
2	12.9	5.036377
3	5.3	1.661683
4	16.0	4.475450

```
# pandas, numpy reused

df = df[(df["fare_amount"] > 0) & (df["fare_amount"] < 200)]
df = df[(df["passenger_count"] > 0) & (df["passenger_count"] <= 6)]
df = df[(df["distance_km"] > 0) & (df["distance_km"] < 100)]

print(df.shape)
```

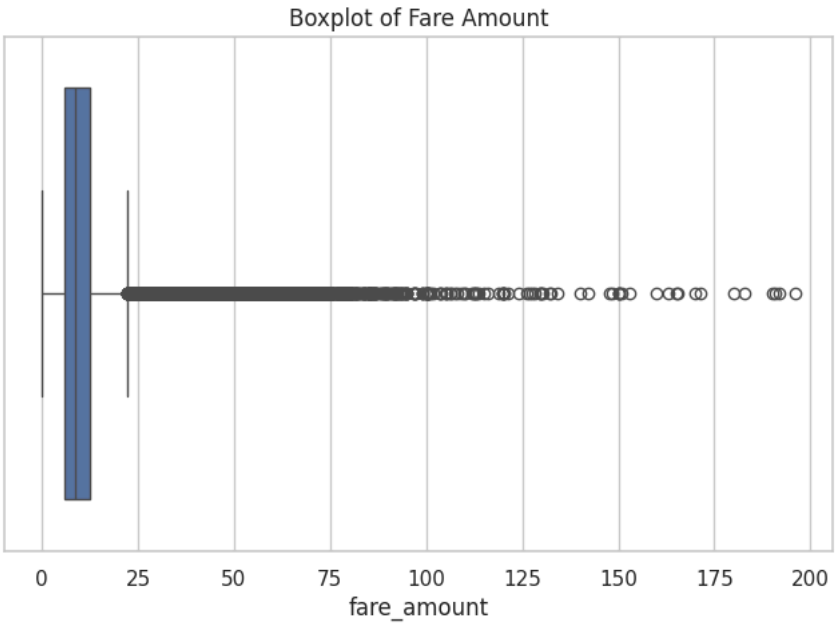
```
(193188, 13)
```

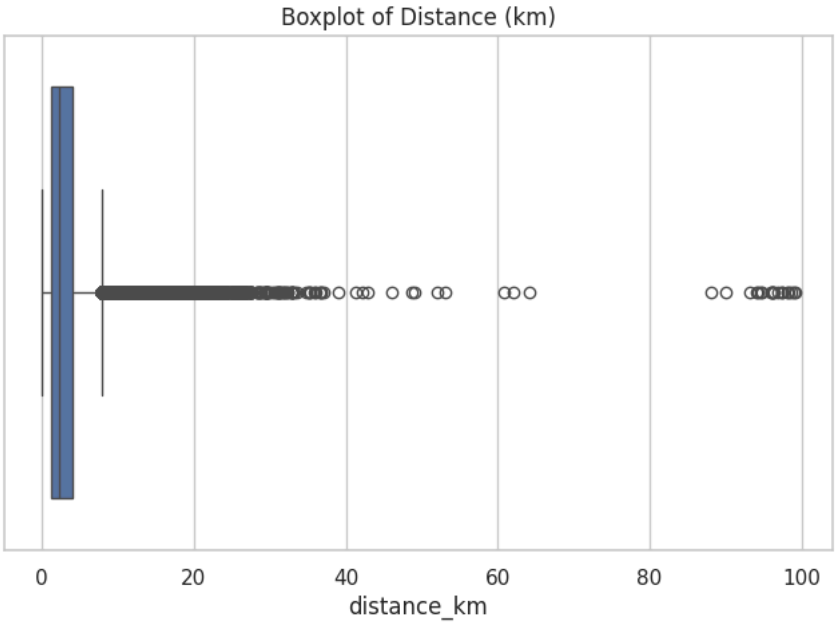
```
import matplotlib.pyplot as plt  # for plotting
import seaborn as sns            # for nicer plots

sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)

plt.figure()
sns.boxplot(x=df["fare_amount"])
plt.title("Boxplot of Fare Amount")
plt.show()

plt.figure()
sns.boxplot(x=df["distance_km"])
plt.title("Boxplot of Distance (km)")
plt.show()
```






```
# pandas, numpy reused

Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers_fare = df[(df["fare_amount"] < lower_bound) | (df["fare_amount"] > upper_bound)]
print("Number of fare outliers:", outliers_fare.shape[0])
```

```
Number of fare outliers: 16536
```

```
# pandas reused

numeric_cols = [
    "fare_amount",
    "pickup_longitude", "pickup_latitude",
    "dropoff_longitude", "dropoff_latitude",
    "passenger_count",
    "pickup_year", "pickup_month", "pickup_day",
    "pickup_hour", "pickup_weekday",
    "distance_km"
]

corr = df[numeric_cols].corr()
```

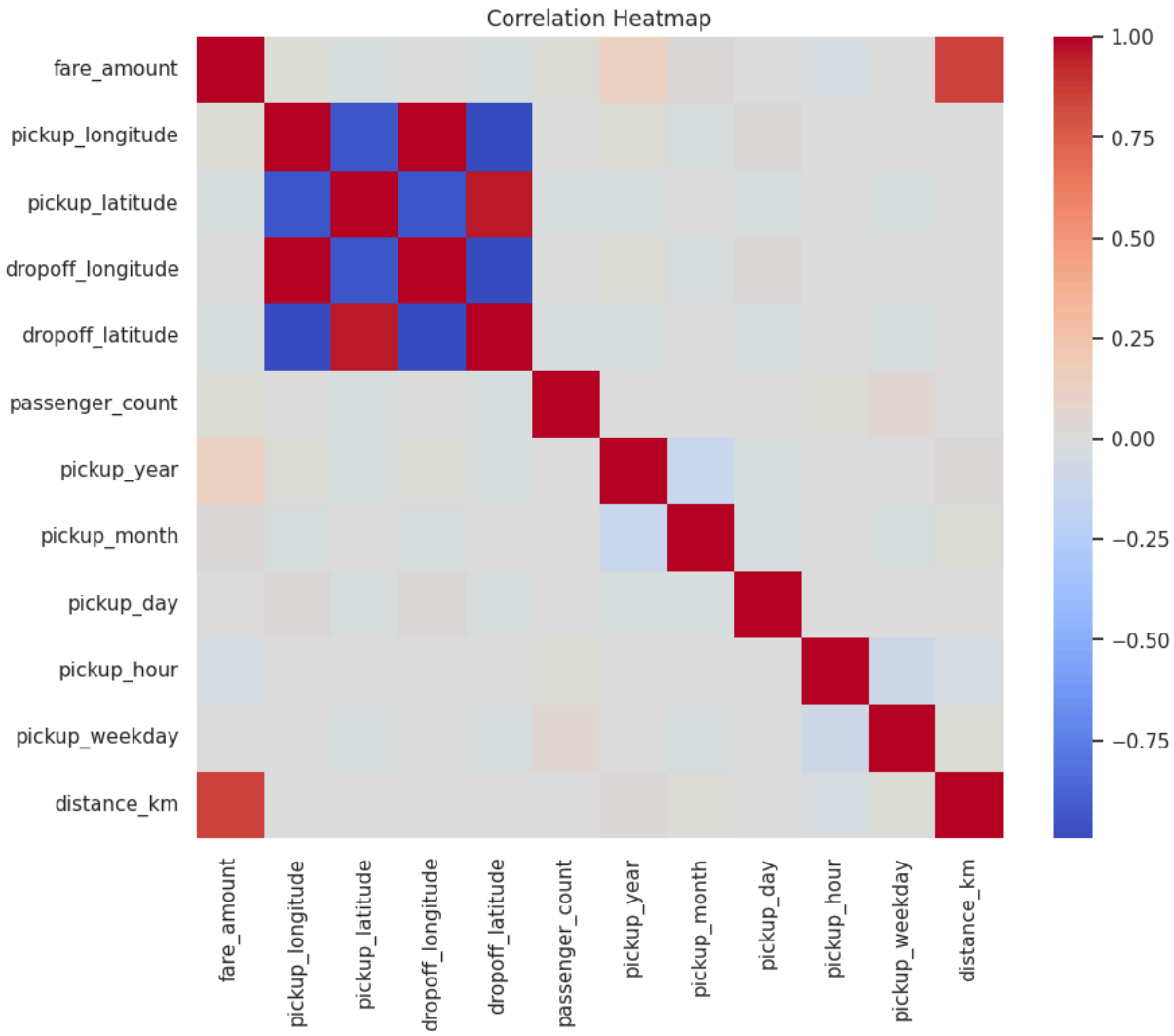
```
corr
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	pick
fare_amount	1.000000	0.011853	-0.008444	0.010396	-0.008448	0.012243	0
pickup_longitude	0.011853	1.000000	-0.949089	0.999879	-0.993968	0.009665	0
pickup_latitude	-0.008444	-0.949089	1.000000	-0.949089	0.954757	-0.009747	-0
dropoff_longitude	0.010396	0.999879	-0.949089	1.000000	-0.993953	0.009661	0
dropoff_latitude	-0.008448	-0.993968	0.954757	-0.993953	1.000000	-0.009789	-0
passenger_count	0.012243	0.009665	-0.009747	0.009661	-0.009789	1.000000	0
pickup_year	0.121490	0.013433	-0.013686	0.013327	-0.014347	0.005316	1
pickup_month	0.024270	-0.007509	0.007618	-0.007471	0.007996	0.008779	-0
pickup_day	0.000499	0.019554	-0.019397	0.019575	-0.020124	0.002600	-0
pickup_hour	-0.020578	0.001840	-0.001822	0.000945	-0.001012	0.014761	0
pickup_weekday	0.005513	0.008242	-0.008916	0.008534	-0.008913	0.036321	0
distance_km	0.847808	0.005550	0.003497	0.004583	-0.001722	0.007392	0

Next steps: Generate code with corr New interactive sheet

```
# matplotlib and seaborn already imported in Cell 8

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=False, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



```
# pandas reused

feature_cols = [
    "distance_km",
    "passenger_count",
    "pickup_hour",
    "pickup_weekday",
    "pickup_month"
]

X = df[feature_cols]
y = df["fare_amount"]

X.head(), y.head()
```

```
(   distance_km  passenger_count  pickup_hour  pickup_weekday  pickup_month
 0     1.683323                1           19               3             5
 1     2.457590                1           20               4             7
 2     5.036377                1           21               0             8
 3     1.661683                3            8               4             6
 4     4.475450                5           17               3             8,
 0     7.5
 1     7.7
 2    12.9
 3     5.3
 4    16.0
 Name: fare_amount, dtype: float64)
```

```
from sklearn.model_selection import train_test_split  # only modeling import here

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape
```

```
((154550, 5), (38638, 5))
```

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

y_pred_lin = lin_reg.predict(X_test)
```

```
from sklearn.linear_model import Lasso  # another model

lasso = Lasso(alpha=0.001, random_state=42, max_iter=10000)
lasso.fit(X_train, y_train)

y_pred_lasso = lasso.predict(X_test)
```

```
from sklearn.linear_model import Ridge  # if not already imported in this session

ridge = Ridge(alpha=1.0, random_state=42)
ridge.fit(X_train, y_train)

y_pred_ridge = ridge.predict(X_test)
```

```
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.linear_model import Ridge
import numpy as np

r2_lin = r2_score(y_test, y_pred_lin)
mse_lin = mean_squared_error(y_test, y_pred_lin)
rmse_lin = np.sqrt(mse_lin)

r2_ridge = r2_score(y_test, y_pred_ridge)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mse_ridge)

r2_lasso = r2_score(y_test, y_pred_lasso)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
```

```
print("Linear Regression  -> R2:", r2_lin,  "RMSE:", rmse_lin)
print("Ridge Regression   -> R2:", r2_ridge, "RMSE:", rmse_ridge)
print("Lasso Regression   -> R2:", r2_lasso, "RMSE:", rmse_lasso)
```

```
Linear Regression  -> R2: 0.7251578128640465 RMSE: 5.023890707868474
Ridge Regression   -> R2: 0.7251578082256225 RMSE: 5.023890750261775
Lasso Regression   -> R2: 0.7251571152292942 RMSE: 5.023897083961173
```

```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge", "Lasso"],
    "R2": [r2_lin, r2_ridge, r2_lasso],
    "RMSE": [rmse_lin, rmse_ridge, rmse_lasso]
})

results
```

	Model	R2	RMSE
0	Linear Regression	0.725158	5.023891
1	Ridge	0.725158	5.023891
2	Lasso	0.725157	5.023897

Next steps: Generate code with `results` New interactive sheet

```
plt.figure()
```